CURSO PARA INTEGRADORES DO HUBSAÚDE · MÓDULO 2 DE 6

# 2 Preparar o ambiente

Todos os exercícios do curso acontecem no seu computador, em uma cópia do HubSaúde feita para testes. Neste módulo, você instala as ferramentas, coloca essa cópia em funcionamento e cadastra nela o seu sistema, do mesmo modo que o credenciamento faz no ambiente oficial.

## AO FINAL, VOCÊ SERÁ CAPAZ DE

- ✓ instalar o HubSaúde CLI e conferir as versões das ferramentas;
- ✓ iniciar, consultar e encerrar o Simulador;
- ✓ gerar um certificado de teste e explicar a diferença entre a parte pública e a chave privada;
- ✓ cadastrar o seu sistema no Simulador com as permissões necessárias.

## COMO O MÓDULO ESTÁ ORGANIZADO

1. **Conceitos:** seções 2.1 a 2.4
2. **Prática guiada:** 7 passos no terminal
3. **Verificação:** perguntas, glossário e respostas

## VOCÊ VAI PRECISAR

- Um computador com Linux, macOS ou Windows, com acesso à internet.
- Um terminal: no Linux e no macOS, o terminal do sistema; no Windows, o PowerShell para a instalação e o Git Bash para os demais comandos.
- Não é preciso permissão de administrador para o HubSaúde CLI.

## ASSISTA AOS VÍDEOS

Aponte a câmera do celular para o código.

SEÇÃO 2.1 E PASSOS 2 E 3

**Instalação e primeiro uso do HubSaúde CLI**

4 min 54 s, com narração e legendas

SEÇÕES 2.3 E 2.4 E PASSOS 5 A 7

**Conhecendo a interface do Simulador**

5 min 49 s, com narração e legendas

## DÚVIDAS E SUGESTÕES

Se algum passo ficar difícil, escreva para o suporte do HubSaúde, em `suporte@saude.go.gov.br`, com o assunto "Curso para integradores, módulo 2". A versão mais recente deste módulo, com os vídeos, está em `https://sesgo-ti.github.io/hubsaude/curso/ambiente/`.

---

Copiar de um PDF pode alterar aspas, hífens e quebras de linha. Para os comandos e arquivos, prefira a versão do módulo no portal ou os arquivos para baixar, em `https://sesgo-ti.github.io/hubsaude/curso/materiais/`.

PARTE 1

# Conceitos

## 2.1 As ferramentas do ambiente de testes

O ambiente de testes reúne três programas. Você interage diretamente apenas com o primeiro, o **HubSaúde CLI**: ele é usado por meio de comandos digitados no terminal e baixa, atualiza e controla os outros dois.

**HubSaúde CLI**

O programa de linha de comando que instala, atualiza e controla as ferramentas. Todo comando começa por `hubsaude`.

**Validador**

Confere se um registro segue as regras dos Guias de Implementação e aponta o que precisa ser corrigido, antes de qualquer envio.

**Simulador**

Uma cópia do HubSaúde que funciona no seu computador, com as mesmas exigências de segurança do ambiente oficial.

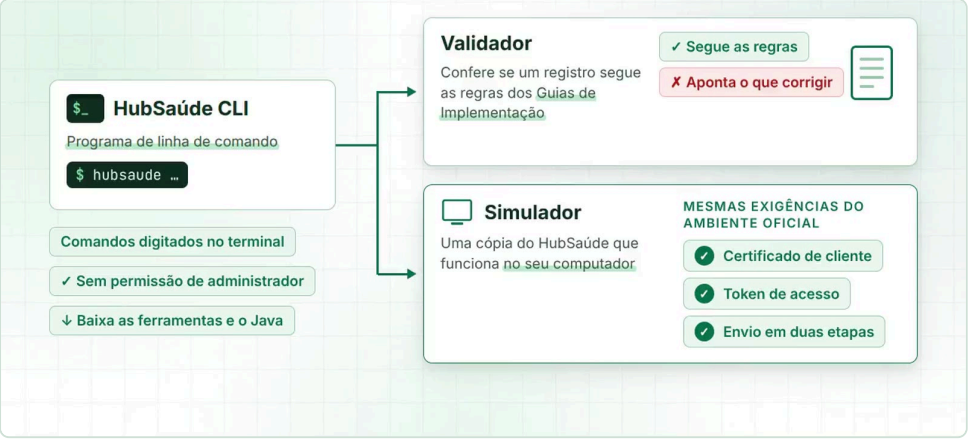



**Figura 2.1.** O HubSaúde CLI controla o Validador e o Simulador. O Simulador exige o certificado, o token de acesso e o envio em duas etapas, como o ambiente oficial. Quadro do vídeo "Instalação e primeiro uso do HubSaúde CLI".

Testar no próprio computador tem duas vantagens. A primeira é a liberdade para errar: os dados são fictícios e nada do que é feito no Simulador chega ao ambiente oficial. A segunda é não depender de ninguém: o integrador começa a trabalhar enquanto o pedido de credenciamento é analisado.

> **DICA**
>
> As ferramentas precisam do Java para funcionar, mas você não precisa instalá-lo: o HubSaúde CLI baixa uma versão própria do Java no primeiro uso, sem alterar o restante do computador.

## 2.2 O certificado de teste

Nos módulos seguintes, o seu sistema vai provar quem é antes de cada envio. A prova é feita com um **certificado digital**, que tem duas partes com papéis diferentes:

**Parte pública**

O certificado propriamente dito, no arquivo `client-cert.pem`. Pode ser compartilhada e é a parte enviada no cadastro. Quem recebe um pedido assinado confere a assinatura com ela.

**Chave privada**

O arquivo `client-key.pem`. Fica apenas com o dono do certificado e é usada para assinar os pedidos. Nunca é enviada a ninguém.

Uma comparação ajuda: a parte pública funciona como um cadeado aberto que você distribui, e a chave privada, como a única chave que o abre. Quem tem o cadeado consegue verificar que a mensagem veio de você, mas não consegue se passar por você.

No ambiente oficial, a instituição usa um certificado ICP-Brasil, emitido por uma autoridade certificadora. No Simulador, você mesmo cria um certificado de teste, sem valor fora do seu computador.

> **ATENÇÃO**
> Nunca use o certificado ICP-Brasil da instituição nos exercícios do curso. O certificado de teste existe justamente para que a chave privada verdadeira não circule por ambientes de teste.

## 2.3 Permissões: os escopos

Cada sistema cadastrado tem uma lista de permissões, chamadas de **escopos**, que indicam o que ele pode fazer. Todo escopo tem três partes, separadas por uma barra e por um ponto:




**Figura 2.2.** A anatomia de um escopo: o prefixo `system`, o tipo de registro e as operações permitidas. Quadro do vídeo "Conhecendo a interface do Simulador".

| Parte | Exemplo | O que indica |
|---|---|---|
| Prefixo | `system/` | O acesso é de um sistema, sem uma pessoa conectada. É o caso de toda integração. |
| Tipo de registro | Patient | O recurso FHIR a que a permissão se aplica. |
| Operações | `.c`, `.rs` | As letras permitidas: `c` para criar, `r` para ler e `s` para pesquisar. |

Os exercícios do curso usam três escopos:

| Escopo | Permite | Usado no |
|---|---|---|
| `system/Patient.c` | Criar registros de paciente | Módulo 5, no envio |
| `system/Patient.rs` | Ler e pesquisar registros de paciente | Módulo 5, na conferência do registro aceito |
| `system/Task.rs` | Consultar o andamento de um envio | Módulo 5, no acompanhamento |

> **OBSERVE**
> Um sistema recebe apenas as permissões de que precisa. Se o escopo não estiver no cadastro, o pedido é recusado, mesmo com o certificado correto.

## 2.4 O cadastro faz o papel do credenciamento

No ambiente oficial, o gestor solicita o acesso no Ganesha e a Secretaria de Estado da Saúde define o `client_id` e as permissões da instituição. No Simulador, não há a quem pedir: você mesmo faz as três coisas.




**Figura 2.3.** À esquerda, o credenciamento no ambiente oficial; à direita, o mesmo resultado obtido por você no Simulador. Quadro do vídeo "Conhecendo a interface do Simulador".

| | Ambiente oficial | Simulador |
|---|---|---|
| **Certificado** | ICP-Brasil da instituição; somente o arquivo `.cer` é anexado | De teste, gerado por você; somente o `client-cert.pem` é enviado |
| **`client_id`** | Devolvido pelo Ganesha após a aprovação | Escolhido por você: `meu-sistema` |
| **Permissões** | Definidas pela Secretaria de Estado da Saúde | Informadas por você no cadastro |

**EM RESUMO**

- O HubSaúde CLI instala e controla o Validador e o Simulador.
- O Simulador é uma cópia do HubSaúde no seu computador, com as mesmas exigências de segurança do ambiente oficial.
- O certificado tem uma parte pública, enviada no cadastro, e uma chave privada, que nunca sai do seu computador.
- Os escopos dizem o que o sistema pode fazer: tipo de registro e operações.
- O cadastro no Simulador faz o papel do credenciamento: define o `client_id`, o certificado e as permissões.

**PARTE 2**

# Prática guiada

Os comandos deste material usam a sintaxe do terminal bash, como no Linux. No Windows, instale o HubSaúde CLI pelo PowerShell e execute os demais comandos no **Git Bash**. As diferenças entre os sistemas aparecem apenas nos passos 1 e 2.

## 1 Instalar os programas de apoio

Garantir que o OpenSSL, o curl e o jq estejam disponíveis no terminal.

Os exercícios usam três programas de apoio:

| Programa | Para que serve no curso |
|---|---|
| OpenSSL | Gerar e consultar o certificado de teste. |
| curl | Fazer requisições ao Simulador pelo terminal. |
| jq | Ler respostas e arquivos em JSON. |

Em distribuições derivadas do Debian, como o Ubuntu:

**VOCÊ DIGITA**

```
$ sudo apt update && sudo apt install openssl curl jq
```

| Sistema | Como instalar |
|---|---|
| Windows | No PowerShell: `winget install --id Git.Git -e --source winget` e `winget install --id jqlang.jq -e --source winget`. O Git for Windows já inclui o Git Bash, o OpenSSL e o curl. Depois, feche e abra novamente o Git Bash. |
| macOS | Com o Homebrew: `brew install openssl jq`. O curl já acompanha o sistema. |

Para conferir, peça a versão de cada programa:

**VOCÊ DIGITA**

```
$ openssl version
$ curl --version
$ jq --version
```

**OBSERVE**

Cada comando deve exibir um número de versão. Se algum responder `command not found`, o programa não foi instalado ou o terminal precisa ser aberto novamente.

☐ **Passo concluído**

## 2 Instalar o HubSaúde CLI

Instalar o programa que controla o Validador e o Simulador.

**VOCÊ DIGITA**

```
$ curl -fsSL https://raw.githubusercontent.com/sesgo-ti/hubsaude/main/install.sh | bash
```

No Windows, use o PowerShell: `irm https://raw.githubusercontent.com/sesgo-ti/hubsaude/main/install.ps1 | iex`.

**VOCÊ VÊ**

```
[i] Plataforma detectada: linux/amd64 (binario: hubsaude-linux-amd64)
[i] Descobrindo a versao mais recente do CLI em sesgo-ti/hubsaude...
[ok] Versao alvo: 0.3.16 (tag hubsaude-cli-v0.3.16)
[i] Baixando hubsaude-linux-amd64...
[i] Baixando checksums.txt...
[i] Verificando integridade (SHA-256)...
[ok] Checksum verificado: 34db814dc004efa44ff11fff60ed770cc7cbc721b6c8fbe164cd4b7d350beec3
[ok] Instalado: /home/usuario/.local/bin/hubsaude
[ok] hubsaude 0.3.16 instalado com sucesso. Experimente: hubsaude --help
```

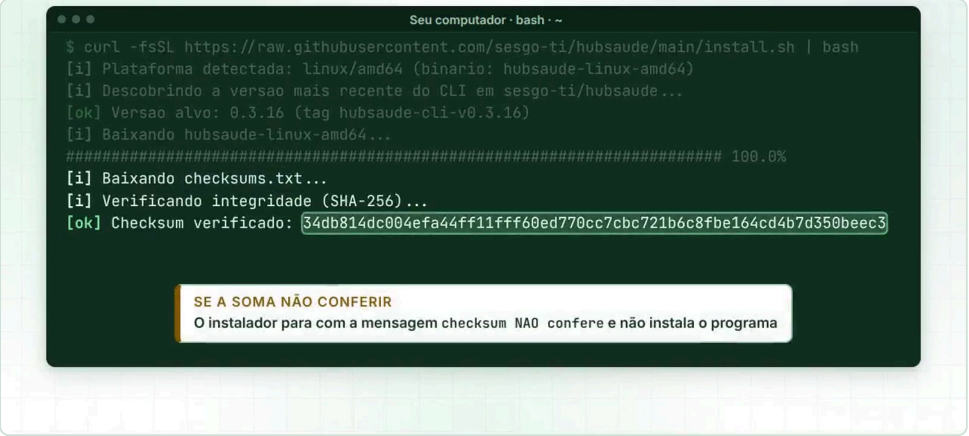



**Figura 2.4.** A conferência de integridade: se a soma não conferir, o instalador para e não instala o programa. Quadro do vídeo "Instalação e primeiro uso do HubSaúde CLI".

**OBSERVE**

Antes de instalar, o instalador baixa a lista oficial de somas de verificação, o `checksums.txt`, e compara com a soma do programa baixado. A soma funciona como uma impressão digital do arquivo: qualquer alteração no caminho muda o resultado. Se o instalador avisar que a pasta de instalação não está no `PATH`, siga a orientação exibida e abra um novo terminal.

Agora, peça as versões do programa e das ferramentas:

**VOCÊ DIGITA**

```
$ hubsaude version
```

VOCÊ VÊ

```
CLI:       0.3.16 (linux/amd64, commit …)
Validador: não provisionado (disponível: 0.1.43)
Simulador: não provisionado (disponível: 0.1.40)
Server:    não provisionado (disponível: 8.10.0)
JRE:       não provisionada
```

**OBSERVE**

"Não provisionado" significa que a ferramenta ainda não foi baixada. Ela é baixada automaticamente no primeiro uso. Os números de versão no seu computador podem ser diferentes dos mostrados aqui.

☐ **Passo concluído**

---

## 3 Iniciar o Simulador

Colocar o Simulador em funcionamento e abrir a sua página no navegador.

VOCÊ DIGITA

```
$ hubsaude sim start
```

Na primeira vez, o programa baixa o Java e o Simulador, o que pode levar alguns minutos.

VOCÊ VÊ

```
JRE Temurin temurin21 100% |████████████████████| (52/52 MB, 79 MB/s)
hubsaude-simulador-0.1.40.jar 100% |██████████████| (87/87 MB, 87 MB/s)
Iniciando simulador (pid 307056) https://127.0.0.1:8443 ...
simulador pronto em https://127.0.0.1:8443
```

Para consultar a situação a qualquer momento:

VOCÊ DIGITA

```
$ hubsaude sim status
```

VOCÊ VÊ

```
simulador: em execução em https://127.0.0.1:8443 (pid 307056)
```

Ao pedir as versões de novo, o Simulador e o Java já aparecem com número:

VOCÊ DIGITA

```
$ hubsaude version
```

VOCÊ VÊ

```
Simulador: 0.1.40 (sha256: …)
JRE:       21.0.12.1+1 (Eclipse Temurin)
```

Abra a página do Simulador no navegador:

VOCÊ ABRE NO NAVEGADOR

```
https://localhost:8443
```

Na primeira visita, o navegador alerta que o certificado não é reconhecido. O alerta é esperado, porque o Simulador usa um certificado próprio, emitido para testes. Aceite-o para continuar.

**ATENÇÃO**

O Simulador guarda os dados apenas na memória. Ao ser encerrado com `hubsaude sim stop` ou ao reiniciar o computador, ele apaga os registros e os sistemas cadastrados. Nesse caso, inicie-o novamente e repita os passos 5 e 6.

☐ Passo concluído

## 4 Criar a pasta de trabalho

Reunir em uma pasta os arquivos produzidos durante o curso.

**VOCÊ DIGITA**

```
$ mkdir curso-hubsaude
$ cd curso-hubsaude
```

**OBSERVE**

Todos os comandos dos próximos módulos devem ser executados a partir dessa pasta. Ao abrir um novo terminal, entre nela de novo com `cd curso-hubsaude`.

☐ Passo concluído

## 5 Gerar o certificado de teste

Criar a chave privada e o certificado que identificam o seu sistema no Simulador.

**VOCÊ DIGITA**

```
$ openssl req -x509 -newkey rsa:2048 -keyout client-key.pem \
    -out client-cert.pem -days 365 -nodes \
    -subj "/CN=meu-sistema/O=MinhaInstituicao/C=BR"
```

| Parte do comando | O que faz |
|---|---|
| `req -x509` | Cria um certificado autoassinado, sem autoridade certificadora: adequado para testes. |
| `-newkey rsa:2048` | Gera uma nova chave privada, do tipo RSA, com 2048 bits. |
| `-keyout` e `-out` | Os nomes dos arquivos da chave privada e do certificado. |
| `-days 365` | A validade: um ano a partir de hoje. |
| `-nodes` | Salva a chave sem senha, para que os programas do curso a leiam sem interação. |
| `-subj` | O titular: o nome do sistema (`CN`), a instituição (`O`) e o país (`C`). |

Durante a geração, o terminal exibe uma sequência de pontos e sinais de mais: é o progresso da criação da chave. Para conferir o resultado, consulte o titular e a validade do certificado:

**VOCÊ DIGITA**

```
$ openssl x509 -in client-cert.pem -noout -subject -enddate
```

**VOCÊ VÊ**

```
subject=CN = meu-sistema, O = MinhaInstituicao, C = BR
notAfter=Oct 29 20:09:53 2027 GMT
```

**OBSERVE**

A data em `notAfter` será um ano depois do dia em que você gerou o certificado.

☐ **Passo concluído**

## 6 Cadastrar o sistema no Simulador

Registrar o seu sistema com o certificado de teste e as permissões do curso.

**VOCÊ DIGITA**

```
$ curl -k -X POST https://localhost:8443/clients/register \
    -F "client_id=meu-sistema" \
    -F "certificate=@client-cert.pem" \
    -F "allowed_scopes=system/Patient.rs system/Patient.c system/Task.rs"
```

| Parte do comando | O que faz |
|---|---|
| `-k` | Aceita o certificado próprio do Simulador, o mesmo que gerou o alerta no navegador. |
| `-X POST` | Envia dados ao endereço de cadastro, `/clients/register`. |
| `client_id` | O identificador do sistema, que faz o papel do `client_id` do credenciamento. |
| `certificate=@client-cert.pem` | Anexa o arquivo da parte pública. O `@` indica que o valor é o conteúdo de um arquivo. |
| `allowed_scopes` | Os três escopos da seção 2.3, separados por espaço. |

A resposta repete os dados do cadastro. Aqui, ela aparece formatada para facilitar a leitura:

**VOCÊ VÊ**

```
{
  "client_id": "meu-sistema",
  "allowed_scopes": ["system/Patient.rs", "system/Patient.c", "system/Task.rs"],
  "certificate": {
    "valid_until": "2027-10-29T20:09:53Z",
    "valid_from": "2026-10-29T20:09:53Z",
    "subject": "C=BR,O=MinhaInstituicao,CN=meu-sistema",
    "issuer": "C=BR,O=MinhaInstituicao,CN=meu-sistema",
    "serial_number": "24e3eefdde13c6281ec05488d5c1d50f50a0c5de"
  }
}
```

**OBSERVE**

O `subject` e o `issuer` são iguais porque o certificado é autoassinado. Somente a parte pública foi enviada; a chave privada permanece no seu computador.

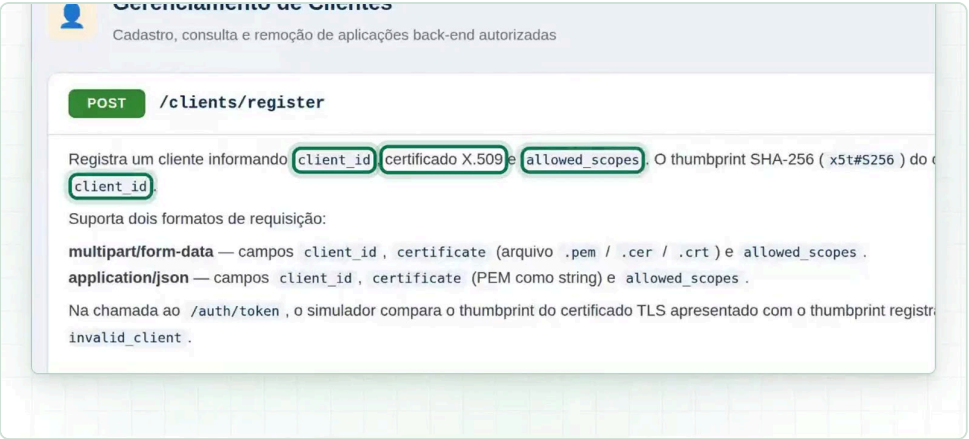



**Figura 2.5.** Na página do Simulador, a seção Gerenciamento de Clientes documenta o mesmo cadastro, com os campos `client_id`, certificado e `allowed_scopes`. Quadro do vídeo "Conhecendo a interface do Simulador".

☐ **Passo concluído**

## 7 Baixar o certificado do Simulador

Obter o arquivo que permite aos seus programas confiar no Simulador.

Até aqui, a opção `-k` fez o curl aceitar o Simulador sem conferir o certificado dele. Os programas dos próximos módulos fazem essa conferência; para isso, precisam do certificado do Simulador.

**VOCÊ DIGITA**

```
$ curl -k https://localhost:8443/certs/simulador-server.pem -o simulador-server.pem
```

Confira o conteúdo da pasta:

**VOCÊ DIGITA**

```
$ ls
```

**VOCÊ VÊ**

```
client-cert.pem  client-key.pem  simulador-server.pem
```

| Arquivo | O que é | Usado no |
|---|---|---|
| `client-cert.pem` | Parte pública do certificado do seu sistema | Módulo 4 |
| `client-key.pem` | Chave privada do seu sistema; nunca é enviada | Módulo 4 |
| `simulador-server.pem` | Certificado do Simulador, para confiar na conexão | Módulos 4 e 5 |

☐ **Passo concluído**

PARTE 3

# Verificação

Responda sem consultar o texto. As respostas estão no final do módulo.

**1.** Qual arquivo é enviado ao Simulador no cadastro do sistema: client-cert.pem ou client-key.pem?

**2.** Você reiniciou o computador e o Simulador deixou de reconhecer o seu sistema. O que aconteceu?

**3.** Por que o cadastro inclui a permissão system/Task.rs?

**4.** O que significa o escopo system/Patient.c?

## Glossário do módulo

**Certificado autoassinado**
Certificado emitido pelo próprio titular, sem autoridade certificadora. Serve apenas para testes.

**Chave privada**
Parte secreta do certificado, usada para assinar os pedidos. Nunca é enviada.

**Escopo**
Permissão de um sistema, no formato `system/Tipo.operações`.

**HubSaúde CLI**
Programa de linha de comando que instala e controla o Validador e o Simulador.

**ICP-Brasil**
Infraestrutura de chaves públicas oficial do país, que emite os certificados usados no ambiente oficial.

**Parte pública**
O certificado propriamente dito, que pode ser compartilhado e é enviado no cadastro.

**Soma de verificação**
Impressão digital de um arquivo, usada para confirmar que ele não foi alterado.

PRÓXIMO MÓDULO

### 3. Montar registros FHIR

Como um registro de saúde é escrito no padrão FHIR, como o Validador aponta os erros e como corrigi-los antes do envio.

# Respostas da verificação

**1.** *Qual arquivo é enviado ao Simulador no cadastro do sistema: client-cert.pem ou client-key.pem?*

O `client-cert.pem`, que é a parte pública. A chave privada, `client-key.pem`, nunca é enviada a ninguém.

**2.** *Você reiniciou o computador e o Simulador deixou de reconhecer o seu sistema. O que aconteceu?*

O Simulador guarda os dados apenas na memória e apagou o cadastro ao ser encerrado. Inicie-o novamente com `hubsaude sim start` e repita o cadastro.

**3.** *Por que o cadastro inclui a permissão system/Task.rs?*

Porque o envio de registros é acompanhado pela consulta de uma Task, o recurso que informa o andamento do processamento. Sem essa permissão, o sistema consegue enviar, mas não consegue saber o resultado.

**4.** *O que significa o escopo system/Patient.c?*

Que o sistema, sem uma pessoa conectada (`system`), pode criar (`c`) registros de paciente (Patient). Ele não permite ler nem pesquisar.